MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

Department of Data Science

BACKEND WEB TECHNOLOGIES HACKATHON (BWT)

PROJECT DOCUMENTATION

Date: 10-10-2026 | Venue: 1st Floor, A & B Blocks

Field	Details
Project Title	Turnin – Assignment Submission Portal
Team Name / Team No.	Team Name: _____ / Team No.: 4
Year / Semester / Section	II Year B.Tech / __1st_ Sem / ___
Technology Stack	Node.js, Express, SQLite, JWT
Programming Language / Tools	JavaScript (Node.js); VS Code
Faculty Mentor	Mrs. V. Nikitha Reddy

Team Members

S.No	Name of the Student	Roll No.	Role
1	A. Sai Charitha	25MVCSDR0005	Team Lead
2	G. Sathvika	25MVCSDR0020	
3	G. Ganesh	25MVCSDR0018	
4	D. Sai Kumar	25MVCSDR0014	

1. Problem Statement

Teachers and students often exchange assignments through WhatsApp, e-mail or paper. This causes lost files, missed deadlines, unclear submission status and delayed marks. Task: Build a secure backend web application where teachers publish assignments, students submit files before the deadline, and teachers grade the work with feedback.

Explanation

- Teachers create assignments with a title, description, deadline and maximum marks.
- Students see every assignment with its status: not submitted, submitted, late, overdue or graded.
- Students upload one file per assignment and can resubmit until it is graded.
- Late submissions are detected automatically by comparing the time with the deadline.
- Teachers view submissions, download files, and give marks with feedback.
- Role-based dashboards show progress, pending work and deadlines, with filters and search.
- Only logged-in users can access data, and each role can use only its own features.

2. Objectives

- Provide secure registration and login using hashed passwords and JWT sessions.
- Enforce role-based access (student / teacher) and ownership checks on every route.
- Validate all inputs and uploads (file-type whitelist, 10 MB limit, marks range).
- Store data in SQLite with relations, allowing one submission per student per assignment.

3. Proposed Approach

- Architecture: single-page frontend (HTML, CSS, vanilla JS) with student and teacher dashboards, calling a REST API built with Express.
- Database: SQLite (better-sqlite3) with three tables: users, assignments and submissions.
- Authentication: bcrypt password hashing; JWT stored in an httpOnly cookie (7-day expiry).
- Authorization: requireAuth and requireRole middleware; teachers access only their own assignments.
- File handling: multer saves uploads with random names; a file is served only to its student or the owning teacher.
- Security: helmet headers, login rate limiting and parameterized SQL queries.

4. Working / Request Flow

1. Register or log in: input is validated, the password is hashed, and a JWT cookie is issued.
2. Every API call passes requireAuth (verify JWT) and requireRole (check role).
3. Teacher creates an assignment: input is validated and saved with the teacher_id.
4. Student submits a file: multer checks type and size, and is_late is set by comparing with the deadline; resubmission replaces the old file unless already graded.
5. Teacher opens the submissions, downloads the file and grades it (0 to max marks) with feedback.
6. Student dashboard shows the status, marks and feedback.

5. API and Database Design

Method	Route	Access
POST	/api/auth/register, /login, /logout	Public
GET	/api/auth/me	Logged in
GET	/api/assignments	Logged in (role-specific data)
POST / PUT / DELETE	/api/assignments[/:id]	Teacher (owner)
GET	/api/assignments/:id/submissions	Teacher (owner)
POST	/api/assignments/:id/submit	Student
GET	/api/submissions/:id/file	Owner student / owning teacher
PUT	/api/submissions/:id/grade	Owning teacher

- Tables: users, assignments, submissions. UNIQUE (assignment_id, student_id) allows one submission per student; ON DELETE CASCADE keeps data consistent.

6. Source Code (Implementation)

- Node.js (JavaScript), Express, SQLite, VS Code.
- Files: server.js, db.js, middleware.js, routes/ (auth, assignments, submissions) and public/ (frontend).

Register and JWT login

JS routes/auth.js

```
11  function sendToken(res, user) {
12    const token = jwt.sign({ id: user.id }, process.env.JWT_SECRET, { expiresIn: '7d'
    });
13    res.cookie('token', token, { httpOnly: true, sameSite: 'lax', maxAge: 7 * 24 *
    3600 * 1000 });
14  }
15
16  router.post('/register', limiter, (req, res) => {
17    const { name, email, password, role, teacherCode } = req.body;
18    if (!name?.trim() || !EMAIL.test(email || '')) return res.status(400).json({
    error: 'Enter your name and a valid email.' });
19    if (!password || password.length < 6) return res.status(400).json({ error:
    'Password must be at least 6 characters.' });
20    const r = role === 'teacher' ? 'teacher' : 'student';
21    if (r === 'teacher' && teacherCode !== process.env.TEACHER_CODE)
22      return res.status(403).json({ error: 'Invalid teacher access code.' });
23    if (db.prepare('SELECT 1 FROM users WHERE email = ?').get(email.toLowerCase()))
24      return res.status(409).json({ error: 'An account with this email already
    exists.' });
25
26    const hash = bcrypt.hashSync(password, 10);
27    const info = db.prepare('INSERT INTO users (name,email,password_hash,role) VALUES
    (?,?,?,?)')
28      .run(name.trim(), email.toLowerCase(), hash, r);
29    const user = { id: info.lastInsertRowid, name: name.trim(), email:
    email.toLowerCase(), role: r };
30    sendToken(res, user);
31    res.status(201).json({ user });
32  });
```

Authentication and role middleware

JS middleware.js

```
5   function requireAuth(req, res, next) {
6     const token = req.cookies.token;
7     if (!token) return res.status(401).json({ error: 'Please log in to continue.' });
8     try {
9       const { id } = jwt.verify(token, process.env.JWT_SECRET);
10      const user = db.prepare('SELECT id, name, email, role FROM users WHERE id =
    ?').get(id);
11      if (!user) return res.status(401).json({ error: 'Account no longer exists.' });
12      req.user = user;
13      next();
14    } catch {
15      res.status(401).json({ error: 'Session expired. Please log in again.' });
16    }
17  }
18
19  // Authorization: are you allowed? (role check)
20  const requireRole = (role) => (req, res, next) =>
21    req.user.role === role ? next() : res.status(403).json({ error: `Only ${role}s can
    do this.` });
```

File upload rules

routes/assignments.js

```
11  const UPLOADS = path.join(__dirname, '..', 'uploads');
12  const ALLOWED = ['.pdf', '.doc', '.docx', '.txt', '.zip', '.ppt', '.pptx', '.png',
    '.jpg', '.jpeg', '.py', '.js', '.java', '.cpp', '.c'];
13  const upload = multer({
14    storage: multer.diskStorage({
15      destination: UPLOADS,
16      filename: (req, file, cb) => cb(null, crypto.randomBytes(16).toString('hex') +
    path.extname(file.originalname).toLowerCase())
17    }),
18    limits: { fileSize: 10 * 1024 * 1024 },
19    fileFilter: (req, file, cb) =>
20      ALLOWED.includes(path.extname(file.originalname).toLowerCase()) ? cb(null, true)
    : cb(new Error('File type not allowed.'))
21  });
```

Submit with late detection and resubmit lock

routes/assignments.js

```
100     const existing = db.prepare('SELECT * FROM submissions WHERE assignment_id = ?
    AND student_id = ?').get(a.id, req.user.id);
101     if (existing && existing.marks !== null) { discard(); return
    res.status(409).json({ error: 'This submission is already graded and locked.' }); }
102
103     const now = new Date();
104     const late = now > new Date(a.deadline) ? 1 : 0;
105     const note = (req.body.note || '').slice(0, 500);
106     if (existing) {
107       removeFile(existing.file_path);
108       db.prepare('UPDATE submissions SET file_path=?, original_name=?, note=?,
    submitted_at=?, is_late=? WHERE id=?')
109         .run(req.file.filename, req.file.originalname, note, now.toISOString(),
    late, existing.id);
110     } else {
111       db.prepare('INSERT INTO submissions
    (assignment_id,student_id,file_path,original_name,note,submitted_at,is_late) VALUES
    (?,?,?,?,?,?,?)')
112         .run(a.id, req.user.id, req.file.filename, req.file.originalname, note,
    now.toISOString(), late);
113     }
114     res.status(201).json({ ok: true, late: !!late });
```

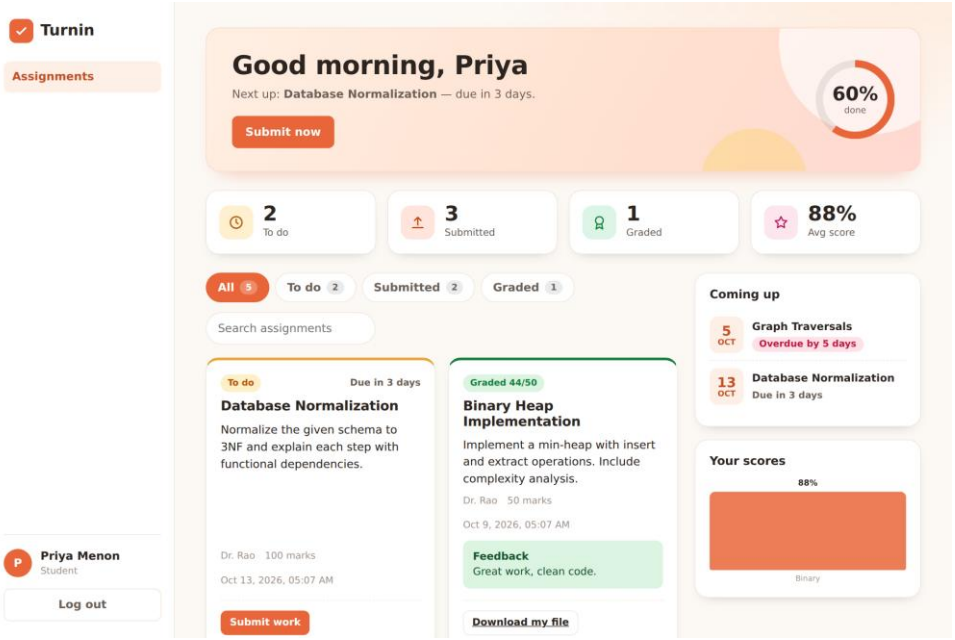
Grading (teacher only, marks validated)

routes/submissions.js

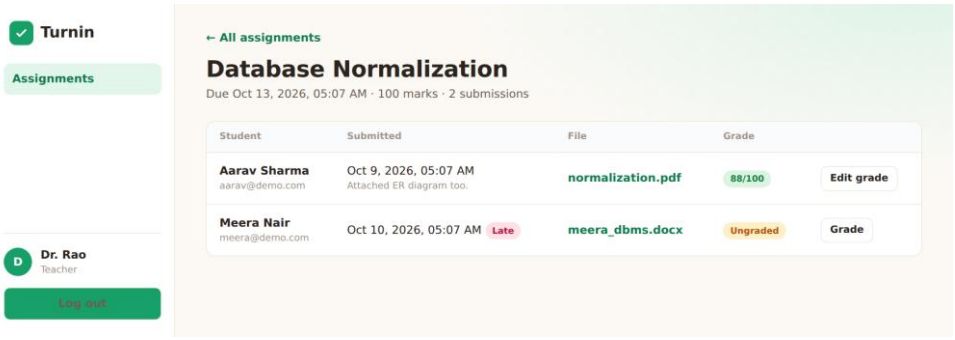
```
18  router.put('/:id/grade', requireRole('teacher'), (req, res) => {
19    const s = db.prepare(`SELECT s.id, a.teacher_id, a.max_marks FROM submissions s
    JOIN assignments a ON a.id = s.assignment_id WHERE s.id = ?`).get(req.params.id);
20    if (!s || s.teacher_id !== req.user.id) return res.status(404).json({ error:
    'Submission not found.' });
21    const marks = Number(req.body.marks);
22    if (!Number.isInteger(marks) || marks < 0 || marks > s.max_marks)
23      return res.status(400).json({ error: `Marks must be a whole number from 0 to
    ${s.max_marks}.` });
24    db.prepare('UPDATE submissions SET marks=?, feedback=?, graded_at=? WHERE id=?')
25      .run(marks, (req.body.feedback || '').slice(0, 1000), new Date().toISOString(),
    s.id);
26    res.json({ ok: true });
27  });
```

7. Results and Test Cases

Student dashboard



Teacher: submissions and grading



- The student dashboard shows a progress ring, summary counts, filters (All / To do / Submitted / Graded) and a status tag on each assignment: To do, Submitted late, Overdue or Graded with marks and feedback.
- The teacher sees each submission with its late tag, file link and grade status.
- Handled safely: wrong password, duplicate email, invalid teacher code, disallowed file type, file over 10 MB, marks out of range, resubmitting after grading, and opening another user's file.

8. Conclusion and Future Scope

- Turnin gives a secure, role-based way to publish, submit and grade assignments in one place.
- Future scope: deadline reminders by e-mail, multiple file uploads, course-wise grouping and grade export to CSV.

9. References

[1] Student notes: Backend Web Technologies (BWT) class notes, Department of Data Science, Malla Reddy Technical Campus.

[2] Express.js documentation, https://expressjs.com.

[3] SQLite documentation, https://www.sqlite.org/docs.html.

[4] MDN Web Docs (HTTP, REST and JavaScript), https://developer.mozilla.org.

[5] JSON Web Token introduction, https://jwt.io/introduction.